

Métodos não paramétricos para regressão

Mineração de Dados

Helton Graziadei

Paramétrico vs. Não Paramétrico

Métodos paramétricos:

- Adotam uma forma funcional para a função de regressão que depende de parâmetros, por exemplo $\psi_{\beta}(\mathbf{x}) = \beta_0 + \beta_1 x$.
- O Número fixo de parâmetros, independente de n .
- Vantagem: fáceis de ajustar, interpretar e testar.
- Desvantagem: se a forma assumida for muito diferente da função de regressão verdadeira, o desempenho preditivo será ruim.

Métodos não paramétricos:

- Não consideram uma forma paramétrica para estimar a função de regressão.
- Mais flexíveis: a “complexidade” do modelo cresce com n .
- Desvantagem: exigem amostras maiores e sofrem com alta dimensionalidade.

O Algoritmo KNN para Regressão

k Vizinhos Mais Próximos (KNN)

A ideia do kNN consiste em obter $\hat{\psi}(\mathbf{x})$ usando a média local das respostas dos k vizinhos mais próximos de $\mathbf{x}$.

Definição

Dado um ponto $\mathbf{x}$ e um inteiro $k \geq 1$, o estimador KNN é

$$\hat{\psi}_k(\mathbf{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_x} y_i,$$

em que $\mathcal{N}_x = \{i \in \{1, \dots, n\} : d(\mathbf{x}_i, \mathbf{x}) \leq d_k^x\}$ e d_k^x é a distância do k -ésimo vizinho mais próximo de $\mathbf{x}$.

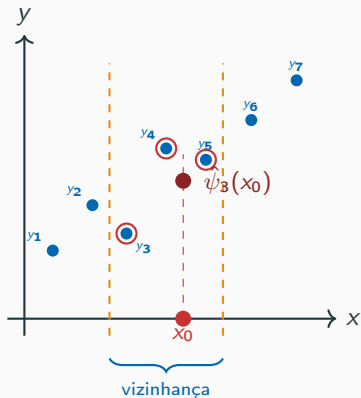
Tipicamente utilizamos a distância Euclidiana.

O algoritmo passo a passo

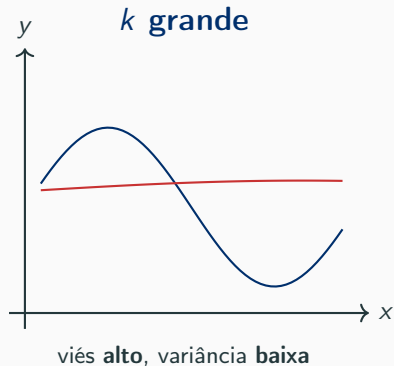
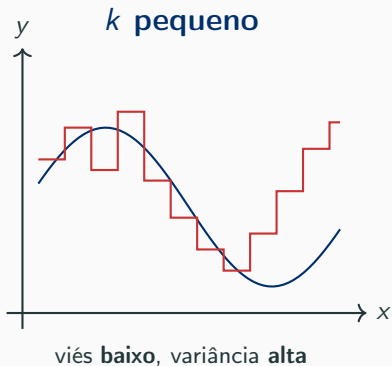
Para prever Y em um novo ponto $\mathbf{x}_0$:

1. Calcule a distância $d(\mathbf{x}_0, \mathbf{x}_i)$ para todo $i = 1, \dots, n$.
2. Ordene as distâncias.
3. Selecione os k vizinhos mais próximos: $\mathcal{N}_x$.
4. Calcule a média das respostas:

$$\hat{\psi}_k(\mathbf{x}_0) = \frac{1}{k} \sum_{i \in \mathcal{N}_{\mathbf{x}_0}} y_i.$$



Influência de k no balanço viés–variância



- $k = 1$: o estimador interpola os dados de treinamento $\Rightarrow$ sobreajuste.
- $k = n$: $\hat{\psi}_n(\mathbf{x}) = \bar{y}$ para todo $\mathbf{x} \Rightarrow$ subajuste.
- O valor ótimo de k é escolhido por validação cruzada.

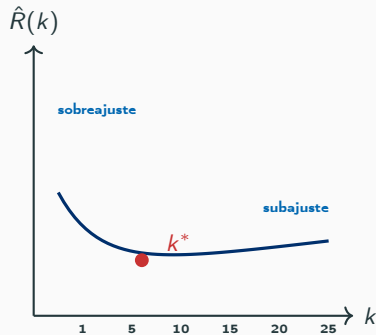
Escolhendo k por validação cruzada

Como escolher k ? Validação cruzada!

Para cada valor candidato de k :

1. Divida os dados em V folds.
2. Para cada fold, ajuste o KNN com k vizinhos nos dados de treino e calcule o erro no fold de validação.
3. Calcule o risco estimado médio $\hat{R}(k)$.

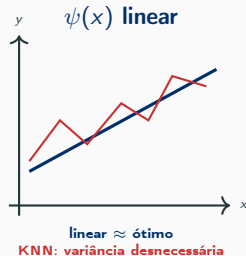
Escolha o k que minimiza $\hat{R}(k)$.



Quando cada método é melhor?

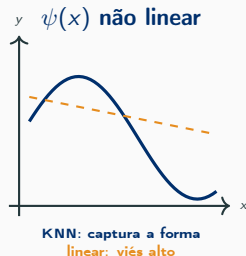
Regressão linear tende a vencer quando:

- A relação verdadeira é (aproximadamente) linear.
- O número de preditoras é grande.
- Poucas observações por preditor.



KNN tende a vencer quando:

- A relação verdadeira é não linear.
- Número de covariáveis é pequeno.
- n é grande o suficiente.



A Maldição da Dimensionalidade

Suponha p preditores independentes, cada um $\sim \mathcal{U}[0, 1]$. Para estimar $f(x_0)$ localmente, o kNN captura uma fração s das observações dentro de um hipercubo centrado em x_0 .

Qual o lado ℓ desse hipercubo?

Para cobrir fração s do volume de $[0, 1]^p$, precisamos de:

$$\ell(s, p) = s^{1/p}$$

p	$s = 1\%$	$s = 10\%$	$s = 50\%$
1	0,010	0,100	0,500
2	0,100	0,316	0,707
5	0,398	0,631	0,871
10	0,631	0,794	0,933
20	0,794	0,891	0,966
50	0,912	0,955	0,986

- Com $p = 10$ e $s = 10\%$, a vizinhança já cobre 79% da amplitude de cada preditor
- Com $p = 50$ e $s = 1\%$, cobre 91% (deixa de ser local).

A Maldição da Dimensionalidade: Consequências para o kNN

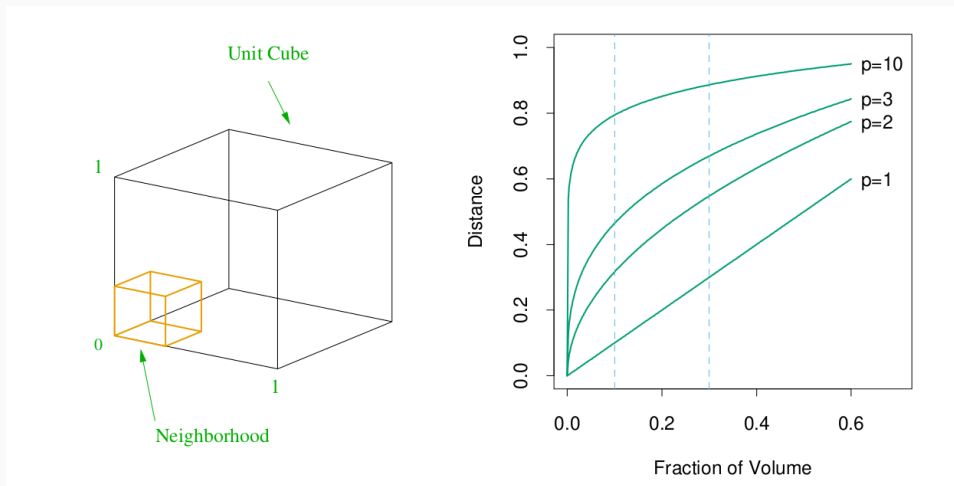


Figure 1: Figura extraída do livro *Elements of Statistical Learning* (2008)

A Maldição da Dimensionalidade: Consequências para o kNN

- $\ell \rightarrow 1$ rapidamente conforme p cresce, para *qualquer* s
- Mesmo capturando apenas 1% dos dados ($s = 0,01$), a vizinhança ocupa quase todo o espaço em p alto
- As três curvas convergem para 1: não há saída

Consequência para o kNN

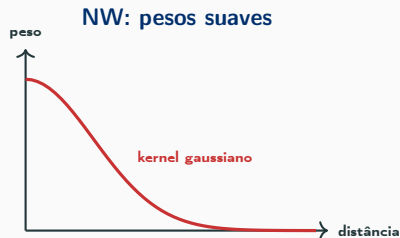
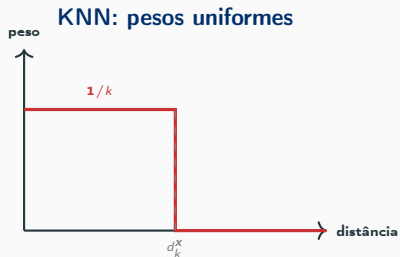
Os k vizinhos mais próximos deixam de ser *locais*. Ou seja, para capturar vizinhos em alta dimensão, a "vizinhança" precisaria abranger quase todo o espaço, perdendo completamente o caráter local do KNN. A média deles não aproxima bem $\psi(x_0) \Rightarrow$ viés cresce, a predição se degrada. Métodos paramétricos contornam isso impondo estrutura em ψ .

Nadaraya–Watson

Nadaraya–Watson: motivação

O KNN atribui peso igual ($1/k$) a cada vizinho e peso 0 aos demais.

E se atribuíssemos pesos diferentes a cada observação, de modo que observações mais próximas recebam peso maior?



Essa é a ideia do estimador de Nadaraya (1964) e Watson (1964).

Nadaraya–Watson: formulação

O estimador de Nadaraya–Watson é uma média ponderada:

$$\hat{\psi}(\mathbf{x}) = \sum_{i=1}^n w_i(\mathbf{x}) y_i, \quad w_i(\mathbf{x}) = \frac{K(\mathbf{x}, \mathbf{x}_i)}{\sum_{j=1}^n K(\mathbf{x}, \mathbf{x}_j)},$$

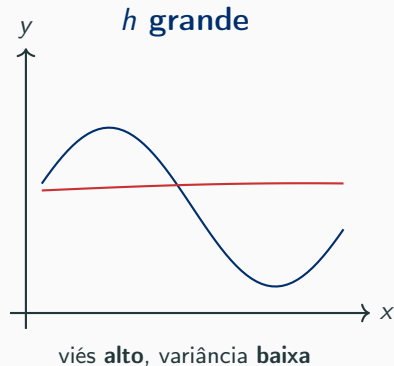
em que $K(\mathbf{x}, \mathbf{x}_i)$ é um *kernel de suavização* que mede a similaridade entre $\mathbf{x}$ e $\mathbf{x}_i$.

Kernels populares:

- **Uniforme:** $K(\mathbf{x}, \mathbf{x}_i) = 1(d(\mathbf{x}, \mathbf{x}_i) \leq h)$
- **Gaussiano:** $K(\mathbf{x}, \mathbf{x}_i) = (\sqrt{2\pi} h)^{-1} \exp\left(-\frac{d^2(\mathbf{x}, \mathbf{x}_i)}{2h^2}\right)$

O parâmetro h é denominada banda. o *tuning parameter* do método.

O papel de h no Nadaraya–Watson

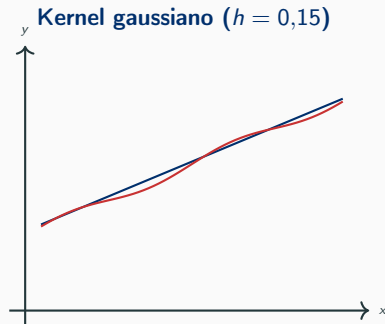
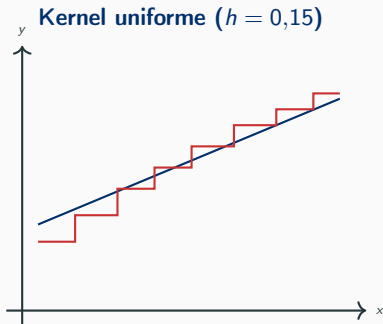


- h pequeno $\Rightarrow$ poucos pontos influenciam $\Rightarrow$ sobreajuste.
- h grande $\Rightarrow$ todos os pontos influenciam igualmente $\Rightarrow$ subajuste ($h \rightarrow \infty$ dá $\bar{y}$).
- h é escolhido por validação cruzada, assim como k no KNN.

Escolha do kernel: importa pouco!

Na prática, a escolha do kernel influencia pouco os resultados.

O que realmente importa é a escolha do tuning parameter h !



NW como mínimos quadrados ponderados

O estimador de Nadaraya–Watson em um ponto $\mathbf{x}$ fixo é a solução de:

$$\hat{\psi}(\mathbf{x}) := \hat{\beta}_0 = \arg \min_{\beta_0} \sum_{i=1}^n w_i(\mathbf{x}) (Y_i - \beta_0)^2.$$

Ou seja: mínimos quadrados ponderados com modelo que contém *somente o intercepto*, ajustado localmente.

Extensão natural — regressão polinomial local (Cleveland, 1979; Fan, 1993):

$$(\hat{\beta}_0, \dots, \hat{\beta}_p) = \arg \min_{\beta_0, \dots, \beta_p} \sum_{i=1}^n w_i(\mathbf{x}) \left(Y_i - \beta_0 - \sum_{j=1}^p \beta_j x_i^j \right)^2,$$

e $\hat{\psi}(\mathbf{x}) := \hat{\beta}_0$. Para cada $\mathbf{x}$, as estimativas dos coeficientes são diferentes.

Vantagem: viés menor, especialmente nas *bordas* do domínio.

Regressão Polinomial

Regressão polinomial: um método não paramétrico?

Já vimos a regressão polinomial:

$$\psi(x) = \sum_{i=0}^{\infty} \beta_i x^i.$$

Na prática, truncamos:

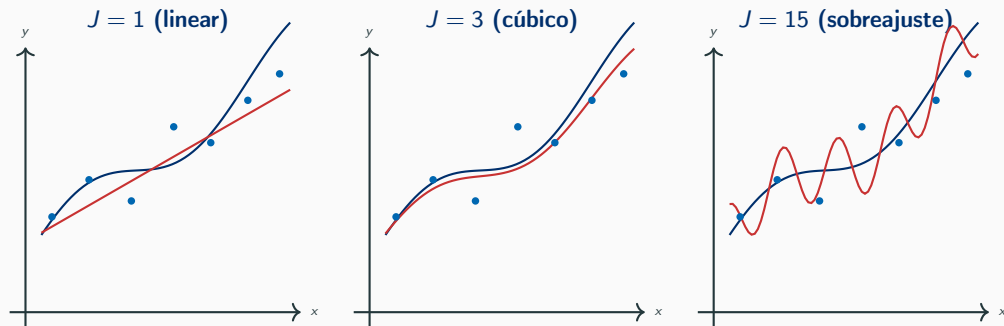
$$\hat{\psi}_J(x) = \sum_{i=0}^J \hat{\beta}_i x^i,$$

e escolhemos J por validação cruzada.

J é o *tuning parameter*: controla a complexidade do modelo.

- J grande $\Rightarrow$ viés baixo, variância alta.
- J pequeno $\Rightarrow$ viés alto, variância baixa.

Regressão polinomial: visualização



O que torna o modelo não paramétrico é que J pode assumir qualquer valor em $\{0, 1, 2, \dots\}$.

Tuning parameters

Todos os métodos que vimos têm tuning parameters que controlam o balanço viés–variância:

Método	Tuning par.	Mais complexo	Menos complexo
Reg. polinomial	J (grau)	J grande	J pequeno
KNN	k (n. vizinhos)	k pequeno	k grande
Nadaraya–Watson	h (banda)	h pequeno	h grande
Lasso	λ (penalização)	λ pequeno	λ grande

Em todos os casos:

- Mais complexo $\Rightarrow$ viés baixo, variância alta.
- Menos complexo $\Rightarrow$ viés alto, variância baixa.
- Escolhemos o tuning parameter por validação cruzada.

KNN — pacote FNN

```
library(FNN)
ajuste <- knn.reg(train = x_treino, test = x_teste,
                  y = y_treino, k = 5)
```

Nadaraya–Watson / Polinomial local — pacote locfit

```
library(locfit)
ajuste <- locfit(y ~ x, alpha = c(0, 0.3), deg = 0,
                 data = dados)
```


1. **KNN:** $\hat{\psi}(\mathbf{x})$ é obtido pela média dos k vizinhos mais próximos.
2. **Nadaraya–Watson:** generaliza o KNN usando pesos suaves via kernel; tuning parameter é a banda h .
3. **Todos compartilham:** um tuning parameter que controla o balanço viés–variância, selecionado por validação cruzada.
4. **Na prática:** a escolha do kernel importa pouco; o que importa é o tuning parameter.